安全科学与工程学院

2026 — 2027 学年第一学期

《互联网应用服务开发与数据库实践》

题目：个人刷题数据分析网站

班级：

学号：241240027

姓名：汪林辰

电子签（必填）：

教师：邢艳

成绩：

2026 年 11 月 5 日

系统设计文档

一 系统需求分析

1.1 系统开发背景

算法竞赛选手的日常训练分散在 Codeforces、AtCoder、洛谷、QOJ 等在线评测平台。各站按自己的规则展示提交历史、竞赛分数和题型标签，数据范围和粒度不一致。选手复盘时，既要分辨多次提交与多道题，也要在中文和英文标签之间手动对照。有的平台只能公开通过记录或累计题量，直接把各站的数字相加，会把重复过题、无日期清单和真实提交混在一起，难以判断某段时间实际练了什么。

1.2 系统目标

项目面向单机个人训练场景，建立一个能从平台账号同步到练习决策的 Web 应用。系统把可获取的题目、标签和判定写入统一数据模型，保留原始来源与统计口径。用户能够从数据简览查看阶段投入，从详细分析查阅题型能力与证据，再进入练习提升页面复盘未过题或挑选难度相近的新题。提交记录保留时间、判定和原题链接，方便逐条核对。

1.3 功能需求

用户可在设置中录入、编辑和解绑外站账号，并主动启动同步。账号可按平台分别配置，QOJ 支持会话凭据，牛客支持数字 UID。总览提供累计题数、AC 题数、最近七天和日历热力图。分析页支持按时间、平台和指标查看练习趋势，同时展示原生竞赛 Rating、题型能力、专职综合能力及历史快照。练习题单支持题型筛选、分页及原题跳转。设置允许归并等价标签、调整技能图谱和选定专职题型。

1.4 非功能需求

同步应允许重试并避免重复计数。写入应维护外键、唯一键和日期约束，网络错误应保留已导入记录。页面采用分页和局部图表更新，避免一次渲染全部历史。能力值须能追溯到入选题目、计分样本和来源。会话凭据以密文存放，浏览器接口只显示配置状态。应用监听本机地址，运行环境、初始化脚本和验收步骤由 README 说明。

二 系统总体设计

2.1 系统功能与模块划分

前端按使用顺序划分为五个页面。设置负责数据入口与偏好，简览和分析负责观察训练状况，练习提升将分析转成可执行题单，提交记录提供原始证据。页面使用 URL hash 确定路由，导航高亮随页面变化。

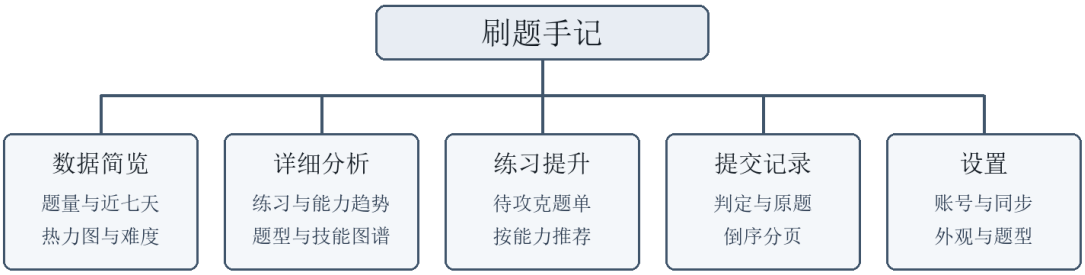


图 1　刷题手记功能模块图

模块	主要行为	典型输出
数据简览	查看累计题量、AC、近七天与全年活动	总量卡片与热力图
详细分析	查看练习趋势、能力快照、平台 Rating 与技能图谱	统计图和可追溯题单
练习提升	按题型复盘未过题或推荐新题	可筛选、分页的训练题单
提交记录	按时间倒序查询真实提交	题目链接、时间与判定
设置	配置外站账号、标签、专职、拓扑和外观	同步状态与用户配置

2.2 层次结构

浏览器使用 Vue 3 组织页面与表单，ECharts 绘制趋势和分布图。Express 同时提供 API 和构建后的静态页面，mysql2 连接 MySQL 8.0。平台适配器通过公开接口或受支持的会话页面读取来源记录，Cheerio 用于解析部分 HTML 页面。

代码按 src/frontend、src/backend 和 src/shared 分层。共享模块保存题型归并与拓扑约束，SQL 和增量迁移位于 sql，测试位于 tests。开发时由 Vite 转发 API，演示时由同一个本机服务提供页面和数据。

2.3 主要流程与控件

账号提交后，Express 先校验账号标识、来源与权限范围，再写入绑定记录。同步请求返回 HTTP 202，任务异步下载并校验数据，随后以事务落库。界面轮询状态，等数据处理结束后显示最终结果。来源失败时可重试，已有历史仍可查询。

用户从题型能力表的「练习」进入题单，路由携带题型键。后端应用用户的标签归并，再计算待攻克集合或筛选 Codeforces 题库。页面通过链接打开题目，后续外站提交在下一次同步后进入分析。

表 1　实际页面控件与用途

控件类型	页面实例	作用
普通文本框	设置中的用户名	录入平台用户标识
密码输入框	QOJ 登录 Cookie	遮蔽输入内容
搜索框	题型选择器	按中英文或题型键匹配
数字输入框	推荐目标难度	输入 CF 尺度目标
月份控件	练习趋势按月筛选	选择统计月份
下拉选择框	能力趋势参考范围	切换当前统计范围
复选框	包含实线下级分支	控制专职范围
范围滑杆	外观字号与卡片间距	即时预览偏好
动作按钮	添加账号或同步账号	提交请求并显示状态
链接	侧栏导航、原题链接	换页或打开原站
展开控件	模块详细说明	按需阅读统计口径
图形节点控件	技能图谱节点	展开分支或查看题型

2.4 状态与数据归属

服务器负责来源数据与图谱配置。外观和侧栏偏好仅保存在当前浏览器。题单分页由后端切片返回，浏览器切换查询时取消旧请求，避免较晚返回的数据覆盖新结果。

三 数据库设计

3.1 概念结构设计

核心实体为用户、来源平台、题目、题型、账号、练习记录和逐次提交。用户在一个平台可配置多个账号，平台下可以有多道题。用户与题目之间用练习汇总表达多对多关系。题型与题目之间通过题型关联表形成多对多关系。

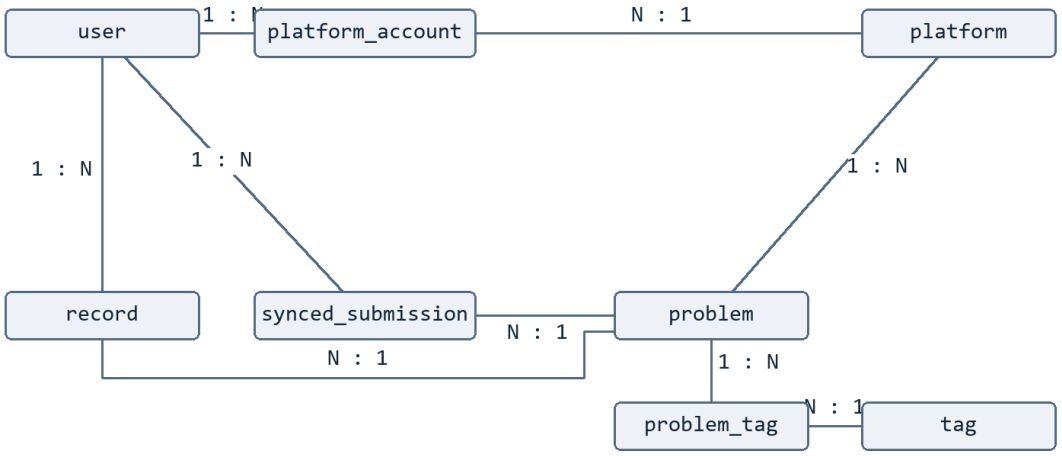


图 2　核心数据实体关系图

扩展实体处理来源粒度和配置版本。练习清单与日期活动分别保存无逐题时间的题目状态和站点每日活动。账号快照及竞赛 Rating 分别挂到平台账号。专职配置和能力快照直接挂到用户。

扩展关系	基数	归属说明
用户与练习清单、日期活动	1 对多	清单另关联题目，活动关联平台
账号与汇总快照、竞赛 Rating	1 对 0 或 1	账号解绑时清理账号附属信息
用户与题型配置	1 对 0 或 1	应用配置与编辑草稿分开
用户与能力快照	1 对多	按照观察顺序保存

3.2 逻辑结构设计

下表以关系名和主键表示逻辑模式。字段含义和类型在物理结构中展开。组合唯一键用于表达来源和业务语义，避免每次同步生成不同的本地主键而绕过业务去重。

表 2　关系模型与关联键

关系	主键	外键及主要约束
user	user_id	用户名称与资料
platform	platform_id	平台名称唯一
problem	problem_id	platform_id，平台与 source_key 唯一
tag	tag_id	题型规范键唯一
problem_tag	problem_id, tag_id	分别关联 problem 和 tag
record	record_id	user_id, problem_id 联合唯一
platform_account	account_id	用户、平台与登录标识联合唯一
synced_submission	submission_id	关联用户、平台、题目，来源提交唯一
platform_problem_inventory	user_id, problem_id	关联用户与题目，记录公开练习清单
platform_daily_activity	user_id, platform_id, account_uid, activity_day	关联用户和平台，按天记录活动
platform_snapshot	account_id	引用平台账号，账号级汇总
account_rating	account_id	引用平台账号，平台原生分
user_skill_settings	user_id	引用用户，保存题型配置及修订号
ability_snapshot	snapshot_id	引用用户，保存观察时间和指标

题目名称、显示标签、平台名称分别在自身表维护。逐次提交保存判定事件，record 提供用户和题目粒度的练习汇总。逐次提交直接关联用户、平台和题目。解绑平台账号后仍能追溯以往练习。

3.3 物理结构设计

表使用 InnoDB 和 utf8mb4。全量初始化统一采用 utf8mb4_0900_ai_ci，以使表列与数据库例程参数的比较规则一致。下表按本机验收库的字段定义列出类型和是否可空。

表 3　用户表　user

字段名称	数据类型	约束	功能说明
user_id	char(36)	主键，非空	所属用户
user_name	varchar(50)	非空	显示名
bio	varchar(500)	可空	简介
avatar	varchar(255)	可空	头像链接

表 4　平台表　platform

字段名称	数据类型	约束	功能说明
platform_id	char(36)	主键，非空	来源平台
platform_name	varchar(50)	非空	平台名称
platform_url	varchar(255)	可空	平台网址

表 5　题型表　tag

字段名称	数据类型	约束	功能说明
tag_id	char(36)	主键，非空	题型标识
tag_name	varchar(120)	非空	规范键
display_name	varchar(180)	可空	显示标签

表 6　题目题型关联表　problem_tag

字段名称	数据类型	约束	功能说明
problem_id	char(36)	主键，非空	题目标识
tag_id	char(36)	主键，非空	题型标识

problem_tag 的联合主键防止同一题目重复挂上相同标签。题型显示名称可为双语文本，业务归并基于规范键。

3.4 题目与练习汇总结构

表 7　题目表　problem

字段名称	数据类型	约束	功能说明
problem_id	char(36)	主键，非空	题目标识
problem_name	varchar(100)	非空	题名
difficulty	ENUM	可空	旧难度档位
platform_id	char(36)	非空	来源平台
source_key	varchar(100)	可空	外站题号
problem_url	varchar(255)	可空	原题链接
native_difficulty	varchar(30)	可空	来源难度
difficulty_source	varchar(40)	可空	评分来源
metadata_checked_at	datetime	可空	元数据时间

表 8　练习汇总表　record

字段名称	数据类型	约束	功能说明
record_id	char(36)	主键，非空	汇总编号
passed	tinyint(1)	非空	通过标记
submit_count	int	非空	汇总次数
solve_date	date	非空	记录日期
user_id	char(36)	非空	所属用户
problem_id	char(36)	非空	题目标识
manual_submit_count	int	非空	手工次数
manual_passed	tinyint	非空	手工通过
manual_solve_date	date	可空	手工记录日期

difficulty 保留早期分类档位。来源评分以 native_difficulty 和 difficulty_source 保存，CF 尺度的折算值由后端计算。manual_* 字段保留原有录入数据，账号同步时与实际账本一起重算汇总。

3.5 账号与提交账本结构

表 9　平台账号表　platform_account

字段名称	数据类型	约束	功能说明
account_id	char(36)	主键，非空	绑定编号
user_id	char(36)	非空	所属用户
platform_id	char(36)	非空	来源平台
login_name	varchar(100)	非空	外站标识
login_pwd	varbinary(16384)	可空	加密会话
last_sync	datetime	可空	上次数据落库时间
remote_uid	varchar(30)	可空	外站数字 UID
sync_status	varchar(20)	非空	同步状态
sync_message	varchar(1000)	可空	同步说明
last_attempt	datetime	可空	最近尝试
sync_cursor	bigint	非空	增量游标

表 10　逐次提交账本表　synced_submission

字段名称	数据类型	约束	功能说明
submission_id	char(36)	主键，非空	本地提交编号
user_id	char(36)	非空	所属用户
platform_id	char(36)	非空	来源平台
source_id	varchar(100)	非空	外站提交号
account_login	varchar(100)	非空	外站用户名
problem_id	char(36)	非空	题目标识
passed	tinyint	非空	通过标记
submitted_at	datetime	非空	提交时间
verdict	varchar(40)	可空	判定文本

login_pwd 保存经过 AES-256-GCM 加密的外站 Cookie。submitted_at 使用北京时间存储，编号在同一提交时刻提供稳定的二次排序。

3.6 公开清单与活动结构

表 11　公开练习清单表　platform_problem_inventory

字段名称	数据类型	约束	功能说明
user_id	char(36)	主键，非空	所属用户
platform_id	char(36)	非空	来源平台
problem_id	char(36)	主键，非空	题目标识
account_uid	varchar(30)	非空	来源账号 UID
passed	tinyint	非空	通过标记
observed_at	datetime	非空	观察时间

表 12　站点日期活动表　platform_daily_activity

字段名称	数据类型	约束	功能说明
user_id	char(36)	主键，非空	所属用户
platform_id	char(36)	主键，非空	来源平台
account_uid	varchar(30)	主键，非空	来源账号 UID
activity_day	date	主键，非空	站点日期
activity_count	int unsigned	非空	活动次数

表 13　账号汇总快照表　platform_snapshot

字段名称	数据类型	约束	功能说明
account_id	char(36)	主键，非空	绑定编号
passed_count	int	非空	已通过数量
attempted_count	int	非空	已尝试数量
activity_days	int	非空	活动天数
observed_at	datetime	非空	观察时间
profile_passed_count	int	可空	主页通过数
profile_attempted_count	int	可空	主页尝试数

洛谷清单只记录可见题目状态，observed_at 是读取时间。站点日期活动独立保存，避免由观察时间推导某道题的提交时间。主页累计值与已获取清单数量分别展示。

3.7 竞赛分与能力快照结构

表 14　竞赛 Rating 表　account_rating

字段名称	数据类型	约束	功能说明
account_id	char(36)	主键，非空	绑定编号
rating	decimal(10,2)	可空	原生竞赛分数
scale	varchar(60)	非空	评分尺度
status	varchar(24)	非空	抓取状态
message	varchar(240)	可空	状态说明
source_url	varchar(255)	可空	评分出处
fetched_at	datetime	可空	获取时间
last_attempt	datetime	非空	最近尝试

表 15　用户题型配置表　user_skill_settings

字段名称	数据类型	约束	功能说明
user_id	char(36)	主键，非空	所属用户
config	json	非空	应用中的配置
revision	int unsigned	非空	当前修订号
updated_at	datetime	非空	更新时间
topology_draft	json	可空	拓扑草稿
draft_revision	int unsigned	非空	草稿修订号

表 16　能力历史快照表　ability_snapshot

字段名称	数据类型	约束	功能说明
snapshot_id	bigint unsigned	主键，非空	自增流水号
user_id	char(36)	非空	所属用户
observed_at	datetime(3)	非空	观察时间
origin	varchar(16)	非空	记录触发来源
metrics	json	非空	分数、样本和范围

竞赛 Rating 和题目难度采用各自的字段与尺度。图谱修订号用于检测并发编辑，拓扑草稿独立保存。能力快照以 UTC 记录观察时间，图表按北京时间显示。

3.8 索引设计与视图实现

表 17　关键索引及用途

数据表	索引	索引字段	作用
record	uk_user_problem	user_id, problem_id	避免同用户同题出现两份汇总
synced_submission	uk_source_submission	user_id, platform_id, source_id	确保来源提交唯一
synced_submission	idx_submission_page	user_id, submitted_at, submission_id	支持稳定倒序分页
problem	uk_platform_source	platform_id, source_key	按平台和外站题号定位
ability_snapshot	idx_ability_snapshot_user	user_id, snapshot_id	按用户读取最近快照

四 系统详细设计与实现

4.1 数据库服务器端

v_problem_progress 使用 UNION ALL 读取有日期的练习汇总和无逐题日期的公开清单，按用户与题目聚合。MAX(passed) 合并通过状态，MAX(solve_date) 只使用已有日期。平台、题型及用户总览继续基于该视图统计。

```
sql/acmer_stats.sql   L777-781
 777  CREATE OR REPLACE VIEW v_problem_progress AS
 778  SELECT user_id,problem_id,CONCAT(user_id,':',problem_id) AS
      record_id,MAX(passed) AS passed,MAX(solve_date) AS solve_date
 779  FROM (SELECT user_id,problem_id,passed,solve_date FROM record
 780   UNION ALL SELECT user_id,problem_id,passed,NULL AS solve_date FROM
      platform_problem_inventory) progress
 781  GROUP BY user_id,problem_id;
```

图 3　练习进度视图的实际 SQL 定义

调用示例为 SELECT COUNT(*) FROM v_problem_progress WHERE user_id = ?。后端的统计接口还会在一致性读事务中查询多个结果，使同一响应使用一致的数据库状态。

4.1.1 存储过程与调用

sp_streak_days 接收用户编号，从最近一次有日期的练习汇总向前逐日查找。只要相邻日期存在记录，连续天数加一，遇到缺失日期即结束。过程在初始化脚本的验证语句中调用。它适合演示数据库例程，页面活动统计则读取逐次提交账本与独立站点活动。

```
sql/acmer_stats.sql   L355-377
CREATE PROCEDURE sp_streak_days(
  IN  p_user_id CHAR(36),
  OUT p_streak  INT
)
BEGIN
  DECLARE v_date DATE;
  DECLARE v_done INT DEFAULT 0;

  /* 最近一次刷题日期作为起点 */
  SELECT MAX(solve_date) INTO v_date FROM `record` WHERE user_id = p_user_id;
  SET p_streak = 0;

  /* 逐日向前回退，遇到没有刷题记录的日期即停止 */
  WHILE v_date IS NOT NULL AND v_done = 0 DO
    IF EXISTS (SELECT 1 FROM `record`
               WHERE user_id = p_user_id AND solve_date = v_date) THEN
      SET p_streak = p_streak + 1;
      SET v_date = DATE_SUB(v_date, INTERVAL 1 DAY);
    ELSE
      SET v_done = 1;
    END IF;
  END WHILE;
END //
```

图 4　连续练习天数存储过程定义

运行时通过 CALL sp_streak_days(用户编号, @streak) 调用，再执行 SELECT @streak 读取输出。库内还提供 sp_add_record，用于 SQL 层录入或累加练习记录。浏览器的增删改查示例使用平台账号与训练题单接口。

4.1.2 触发器与事务

trg_record_check_update 在更新 record 前检查刷题日期和提交次数。如果日期晚于当前日，或者提交次数超出 1 至 999，触发器使用 SQLSTATE 45000 中断写入。新增记录使用对应的插入校验，另以唯一键约束用户与题目。

```
sql/acmer_stats.sql   L533-543
 533  CREATE TRIGGER trg_record_check_update
 534  BEFORE UPDATE ON `record`
 535  FOR EACH ROW
 536  BEGIN
 537    IF NEW.solve_date > CURDATE() THEN
 538      SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = '刷题日期不能晚于今天';
 539    END IF;
 540    IF NEW.submit_count < 1 OR NEW.submit_count > 999 THEN
 541      SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = '提交次数必须在 1~999 之间';
 542    END IF;
 543  END //
```

图 5　更新练习记录前的触发器定义

同步事务及回滚

同步适配器先整理平台数据，再在事务中写入提交、更新题目元数据并重算汇总。同一来源提交在重试时沿用已有账本行，判定变化可更新。事务失败会回滚本次写入。触发器限制的是汇总表的次数范围，逐次提交账本独立保存真实事件。

用例	数据库行为
未来日期	拒绝写入并返回触发器错误
计数小于 1 或大于 999	拒绝更新练习汇总
重复提交来源编号	同步重用账本行并核对判定
首次同步中途发生写入错误	回滚本次事务，保留先前数据

4.2 应用服务器端

4.2.1 数据录入

在「设置」的「平台账号与同步」选择来源、填写用户标识并点击「添加账号」。表单通过 POST /api/accounts 发送数据，后端校验来源与账号格式，加密可选 Cookie 后插入 platform_account。重复绑定返回冲突。

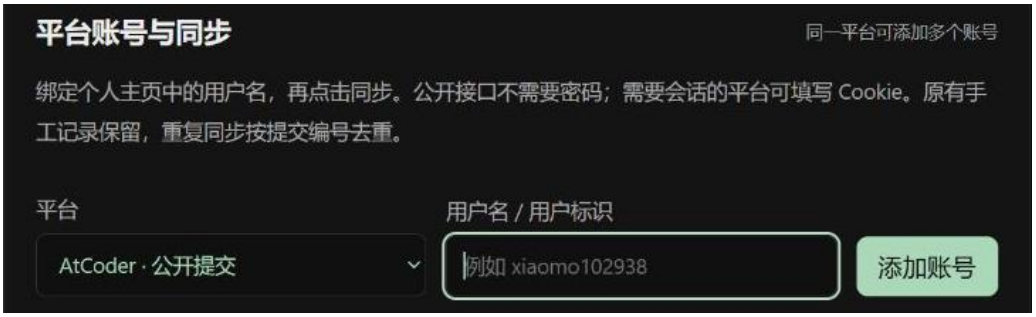


图 6　账号录入界面

```
src/backend/accounts.js   L44-47
  44    const id=randomUUID();
  45    try {await pool.execute('INSERT INTO
      platform_account(account_id,user_id,platform_id,login_name,login_pwd,remote_uid)
       VALUES(?,?,?,?,?,?)',[id,user,req.body.platform_id,login,encrypt(cookie),uid||n
      ull]);}
  46    catch(e){if(e.code==='ER_DUP_ENTRY') throw Object.assign(new
      Error('该账号已经绑定。'),{status:409});throw e;}
  47    res.status(201).json({account_id:id,message:'账号已保存。'});
```

图 7　新增绑定的服务端代码

4.2.2 数据删除

点击「解绑」后页面展示确认区。确认后调用 DELETE /api/accounts/:id，服务端按账号编号与当前用户删除绑定。该操作保留已经同步的练习和提交。



图 8　解绑前的确认界面

```
src/backend/accounts.js   L59-62
  59   router.delete('/:id',async(req,res)=>{
  60    if(busy.has(req.params.id)) throw Object.assign(new
      Error('账号正在同步，请稍后解绑。'),{status:409});
  61    const [result]=await pool.execute('DELETE FROM platform_account WHERE
      account_id=? AND user_id=?',[req.params.id,await owner()]);
  62    res.status(result.affectedRows?200:404).json({message:result.affectedRows?'已解绑
      ，历史刷题记录保留。':'账号不存在。'});
```

图 9　删除绑定的服务端代码

4.2.3 数据检索与结果展示

在「练习提升」选择「待攻克」和「我的专职题型」，前端通过 GET /api/practice 发送模式、题型及页码。后端先读取数据库中的已通过和标签关联，再排除已有 AC 的同源题，按最近尝试去重，返回当前页。



图 10　按专职范围查询待攻克题目

```
src/backend/practice.js   L87-90
  87    if(mode==='retry'){
  88     const filter=selected==='@specialty'?specialtyKeys(config):known?new
      Set([selected]):null;
  89     return res.json({...paginate(retryProblems(latest,progress,tagRows,filter),Nu
      mber(page)),
  90      options,selected:filter?selected:'',hasSpecialties:config.specialties.length
      >0});
```

图 11　待攻克查询的筛选与返回代码

4.2.4 数据修改

在账号条目点击「编辑」可更新会话配置或清除旧 Cookie。页面使用 PUT /api/accounts/:id，保存时校验当前用户与账号。编辑保留绑定标识和已有历史。



图 12　已绑定账号的编辑表单

```
src/backend/accounts.js   L55-57
  55    await pool.execute('UPDATE platform_account SET remote_uid=?,login_pwd=CASE
      WHEN ? THEN NULL WHEN ? IS NOT NULL THEN ? ELSE login_pwd END WHERE account_id=?
       AND user_id=?',
  56     [uid||null,req.body.remove_cookie===true,
      cookie?1:null,encrypt(cookie),req.params.id,user]);
  57    res.json({message:'账号设置已更新。'});
```

图 13　更新账号配置的服务端代码

4.2.5 安全与关键前端代码

安全性实现

写接口校验本机 Origin 和自定义请求头。数据库操作使用参数占位符，Cookie 采用 AES-256-GCM，密钥从本机 .env 读取。页面使用文本插值显示外站返回内容。账号列表只返回是否已配置凭据。当前是单机使用模式。

HTML 结构与表单

```
src/frontend/accounts.js   L60-62
  60    <form ref="formEl" @submit.prevent="save" class="account-form">
  61     <label>平台<select v-model="form.platform_id" :disabled="!!editing"
      required><option v-for="p in platforms" :key="p.platform_id"
      :value="p.platform_id">{{p.platform_name}} ·
      {{p.mode}}</option></select></label>
  62    <label>{{selected()?.platform_name==='洛谷'?'数字 UID':'用户名 / 用户标识'}}<input
      v-model.trim="form.login_name" :readonly="!!editing" maxlength="100" required
      placeholder="例如 xiaomo102938" autocomplete="off"></label>
```

图 14　Vue 模板绑定账号录入表单

表单使用 @submit.prevent 调用保存方法，v-model 把选择值和文本输入映射到表单状态。编辑时平台和账号标识被锁定。

CSS 状态与布局

```
src/frontend/style.css   L127-129
 127  .practice-tabs [aria-pressed="true"]{background:var(--selected);font-weight:600}
 128  .practice-filters{margin-bottom:12px}
 129  .practice-panel .table-wrap{margin-top:16px}
```

图 15　题单标签按钮的选中样式

CSS 使用 aria-pressed 区分当前题单标签，背景色从主题变量取得。卡片间距使用外观变量控制，主题切换只过渡相关颜色。

JavaScript 路由

```
src/frontend/main.js   L136-141
 136    function hashChanged(){
 137     const next=resolvePage();page.value=next;
 138     if(next==='practice'){practiceTag.value=readPracticeTag();return;}
 139     if(location.hash!=='#'+next)history.replaceState(null,'','#'+next);
 140    }
 141    watch(difficultyPlatform,()=>nextTick(draw));
```

图 16　根据地址更新当前页面

切换地址后更新当前页面及推荐题型，旧锚点统一到现行路由。图表在离开页面时释放。

4.3 能力统计与技能图谱

计分方法

题型能力对同一题型中已通过且有可靠评分依据的题目去重，取最高 25 道的算术平均。综合能力默认限制在专职题型范围，可沿实线纳入下级题型，也能切换全部题目。综合计分最多取 150 道。可识别的跨站同源题和重复标签只计一次，样本不足时标为暂定。

表 18　跨平台难度来源

来源	计分方法
Codeforces	采用官方题目 rating。可确认的洛谷 CF 镜像优先使用同源官方分。
AtCoder	采用 AtCoder Problems 社区难度，经现有规则映射到 CF 尺度，显示为估分。
洛谷	以八个难度档位映射为 800、1000、1400、1800、2000、2200、2700、3200。
其他来源	保留题目和提交。有可靠评分依据的题目才进入能力样本。

AT 映射为 $3900 \times (\text{AT 难度} + 940) \div 5155$，限制在 800 到 3900，再按 100 分取整。这套规则将选手评分关系外推到题目，属于当前版本的近似。洛谷采用档位经验值，新增提高档使用 2000。两类估分均保留来源和低置信性质。

题型与技能图谱

标签在统计前经过内置双语映射和用户归并。技能图谱用实线表示主前驱、虚线表示副前驱。根节点无前驱，其余节点只有一个主前驱，可有多个副前驱。保存时校验整个有向图无环。图谱节点题数累计主前驱树的下级通过题，同一题沿多个标签命中时去重。

分析图谱只在展开或收起时更新布局，稳定后停下。编辑器使用独立布局，允许拖动节点、连线和保存草稿，并校验节点重叠。标签归并的预览检查拓扑冲突，处理后才应用配置。

4.4 练习题单与能力变化趋势

待攻克与按能力推荐

待攻克题单先在来源账本按题目取最后一次非 AC，再按规范题目标识去重，并排除已知通过题目。专职筛选使用已保存的题型范围。洛谷清单和只公开 AC 的数据源可以帮助判定已有通过，但缺少失败提交时无法生成相应复盘项。

推荐题单使用缓存的 Codeforces 题库。题目需命中所选题型，与目标难度之差不超过 200 分，并且在本地没有已知练习记录。系统默认参考该题型能力，允许用户手动输入目标分数，按分差排序返回。结果显示原题链接和难度差，用户可从题型分析页直接进入。

能力历史快照

服务启动、同步成功或保存题型设置后，系统复用能力计算函数生成快照。在分数、样本或计分范围变化时才写入。快照保存全部题目综合、专职综合及各选定专职题型的分数、样本量和计分定义。计分范围或评分版本变化时折线断开，避免把口径变化读成能力涨跌。

表 19　分析与练习接口

方法与路径	数据或用途
GET /api/dashboard	总览、分布、能力参考和技能图谱
GET /api/activity	热力图和站点日期活动
GET /api/trends	按时间范围和粒度汇总提交
GET /api/submissions	逐次提交倒序分页
GET /api/ability/overall	综合能力及前 150 题证据
GET /api/tags/:id/ability	单个题型的前 25 题证据
GET /api/practice	待攻克和推荐题单
GET /api/ability/trends	最近能力快照及计分口径

能力快照从功能启用时开始累积。历史练习趋势以已有提交日期计算，二者回答的问题不同。

4.5 系统测试与验收

测试分为规则单元测试、隔离数据库测试、HTTP 与数据库结果核对、真实来源验收。完整建库和重复同步在独立临时库执行，日常练习库用于只读核对。

表 20　核心验收结果

检验项	执行与结果
初始化和启动	全量脚本在独立 MySQL 库成功执行。本机首页与核心 API 返回 HTTP 200。
账号 CRUD	测试覆盖绑定、重复拒绝、编辑、解绑、凭据隐藏及来源校验。
真实重复同步	同一 Codeforces 账号在临时库连续同步两次，均为 641 条提交。
训练与能力快照	测试库生成 28 道待攻克题。手动目标为 1600 分的 DP 推荐返回 468 道。读取两条能力快照。
趋势与分页	用数据库账本核对热力图、首次 AC、时间桶、前两页顺序和越界页。
评分口径	核对最高 25 与 150 题均值、同源去重及估分数量。
失败回滚	隔离库验证重判、手工记录保留、清单幂等及事务回滚。
构建与回归	Vite 构建、Windows 启动前置检查及 71 项自动化测试通过。

验收发现及处理

在全新建库时，原脚本的例程参数与表列使用了不同排序规则，示例调用中断。现统一建库和表的 utf8mb4 排序规则。重复同步验收还发现后台收尾期间页面提前显示成功，已将最终状态推迟到数据处理与 Rating 刷新结束。两项修复均通过复测。

真实外站重同步这轮覆盖 Codeforces。各站公开数据范围和会话条件不同。大规模负载、长期同步及不同设备的兼容性仍需另行验证。

五 系统说明

5.1 系统开发环境

项目	环境与说明
运行系统	Windows 本机环境
应用运行时	Node.js 22.12 或更新版本，npm
数据库	MySQL 8.0，InnoDB，utf8mb4
前端	Vue 3、ECharts、Vite
后端	Express、mysql2、Cheerio
版本控制	Git，GitHub 远程仓库

5.2 安装配置与本机运行

1. 克隆项目仓库，在项目根目录执行 npm ci。首次建库在 MySQL 中执行 sql/acmer_stats.sql。脚本会删除并重建同名 acmer_stats 库。已有数据库先备份，再按序应用尚未执行的迁移，当前最新为 013。

2. 复制 .env.example 为 .env，填写本机数据库参数。需要保存外站 Cookie 时生成 64 位十六进制密钥，并写入 ACCOUNT_SECRET_KEY。已保存凭据依赖同一密钥解密。

3. 执行 npm run build 后执行 npm start，浏览器访问 http://127.0.0.1:3000/。Windows 也可运行 start-site.cmd。停止服务时，在启动终端按 Ctrl+C。

5.3 演示顺序

从设置绑定 Codeforces 并触发同步，观察状态变化。到数据简览查看热力图，再在提交记录切换页码并打开原题。到详细分析切换练习趋势或能力快照，点击题型表的「练习」进入推荐题单，最后查看待攻克和专职筛选。

提交包按指导书使用「学号姓名」命名，包含本人签字后的报告 Word、与之对应的 PDF、程序源代码和数据库脚本或备份。课程示例和数据库结构可用仓库脚本重建。

六 课程设计总结

本次课程设计把个人刷题统计从数据库练习扩展为一套可操作的 Web 应用。最初的模型只有题目、标签和练习汇总，接入外站后，同一道题可能出现多条提交、多个标签或多种来源。若直接在汇总表累加，重复同步会让题数和提交数失真。我把逐次提交、公开练习清单与每日活动分开保存，再用统一视图按用户和题目组织进度。这使我能在页面上追溯数字，也能用数据库测试验证回滚和重试。

实现中另一个难点是不同平台难度和题型不可直接比较。双语标签归并解决了一部分名称差异，能力参考选择最高难度样本，并让用户查看哪些题参与了计算。这个过程让我理解到，统计界面的一个数字同时依赖数据来源、范围、去重和计算公式。能力快照增加了一个时间维度，也要求在专职范围改变时保留当时的口径。待攻克和推荐题单进一步把统计结果接回到练习过程。

验收时，从已配置的个人库运行正常并不能证明老师能从零安装。通过独立建库，我发现排序规则不一致会使存储过程示例失败。连续两次同步也暴露了后台收尾与前端状态不一致。把这些问题纳入复测后，部署说明和测试结果能够对应真实操作。现阶段仍受外站公开数据覆盖、会话有效期和估分误差影响。后续可在自愿参与和数据脱敏的前提下验证跨平台校准，进一步检验推荐难度是否适合实际训练。

七 AI 使用说明

初稿曾使用 Kimi 辅助梳理需求和数据库结构。开发中使用 Codex 辅助代码实现、排错、测试与文档整理。报告中的代码图取自仓库实际源码，界面图来自本机运行。个人署名与电子签由作者填写。答辩前，作者需按指导书复核代码、运行流程和统计口径，并能够解释或修改关键逻辑。

八 版本控制记录

代码通过 Git 保存逐步变更。下表按功能选取可以在本机仓库查询的提交。完整版本记录可用 git log --oneline 查看。

表 21　代表性 Git 提交

提交	变更
1355ac8	增加主题切换及系统外观跟随
196c76e	增加年度热力图及多维活动筛选
af47b37	归并双语标签并计算题型能力
91949aa	接入洛谷公开练习清单与站点活动
d11d88b	实现待攻克和按能力推荐接口
adb8a85	新增练习提升页面及分析入口
23ebb53	保存能力历史快照
d30e3fb	在详细分析展示能力变化趋势
064b1f8	统一全新建库脚本的排序规则
03b4b61	后台处理完成后再显示同步成功
ac97e21	整理安装指南与验收报告

参考资料

[1] Whalica. OJ Insight. https://github.com/Whalica/OJ_Insight

[2] Codeforces. Codeforces API. https://codeforces.com/apiHelp

[3] kenkoooo. AtCoder Problems. https://github.com/kenkoooo/AtCoderProblems

[4] 洛谷. 题目难度. https://help.luogu.com.cn/manual/luogu/problem/difficulty

[5] Silver Fox. AT–CF Rating Converter. https://silverfoxxxy.github.io/rating-converter

[6] 本项目仓库. https://github.com/xiaomo102938/acmer-stats